Resilience-Alpha – Debrief et répartition

Oct 19, 2026 · @Kevin Ferraretto

Le but : une plateforme de paiement fictive, conteneurisée, qui se répare toute seule après une attaque lancée depuis un second PC. Le jury veut voir le service avant, pendant et après l'incident, avec une nouvelle transaction réussie à la fin.

Ce qu'on doit livrer

Les 5 piliers sont obligatoires :

1. **Dev** : microservices (BDD, API, auth, front), moteur de remédiation, dashboard React/Tailwind.
2. **Infra** : Docker Compose, Ansible, Prometheus, Grafana, Loki (pas d'Elastic).
3. **IA** : détection d'anomalies en ML (pas de règles en dur) et assistant RAG Ollama + LangChain (modèle de 8B paramètres maximum).
4. **Cyber** : attaque réelle depuis le PC Attaquant, WAF, SIEM ou fonction SOC, durcissement, Vault ou SOPS.
5. **Gouvernance** : EBIOS RM, PCA/PRA, analyse NIS2/DORA reliée à notre architecture réelle.

Rendus (dossier `Workshop2026-M2-G<n>`) :

Fichier	Contenu
`-Dossier.pdf`	8 à 12 pages, contribution de chacun visible
`-Pres.pptx`	Support court devant des professionnels
`-VidChrono.mp4`	60 s maximum, sans coupe
`-Code.zip`	Sources, README, aucun secret en clair

Les points qui font gagner ou perdre

Une intervention humaine pendant le live donne 0/5 sur l'axe remédiation, le plus gros axe de la grille locale.

- Un conteneur qui redémarre ne prouve rien : il faut vérifier l'API **et** une transaction.
- Le ML doit être branché sur la remédiation, pas seulement présenté à côté.

- Le moteur de remédiation a des droits élevés : aucune interface d'admin exposée. Prévoir un docker-socket-proxy ou une API restreinte avec token.
- L'assistant RAG est distinct de l'automate et n'a aucun accès privilégié.
- Il faut distinguer restauration de disponibilité et correction d'une compromission (par exemple pour l'exfiltration).
- Chacun doit pouvoir expliquer le code généré par IA devant le jury.
- Sécurité : aucune attaque hors du labo, aucun secret donné à une IA.

Notation locale : (suivi individuel + 2 × note du jury) ÷ 3. Le jury donne 10 minutes : 5 de démo, 5 de questions.

Dates clés : architecture et EBIOS validés lundi, gel du code jeudi à 17h, remise avant 23h59 jeudi, jury vendredi.

Répartition proposée

Cinq personnes, un rôle principal chacun, avec Hugo seul sur l'infra et la cyber.

Qui	Rôle principal	Responsabilités
Hugo	Infra + Cyber	Docker Compose, playbooks Ansible, Prometheus/Grafana/Loki, WAF (ModSecurity ou Coraza), SIEM (alertes Loki/Grafana, ou Wazuh si la RAM suit), Vault/SOPS, durcissement, hotspot Windows et réseau
Kevin	Tech lead + remédiation	Architecture globale, moteur de remédiation (décision, action, vérification), ML d'anomalies (jeu de données, modèle, branchement sur la chaîne), cohérence d'intégration
Antoine	Dev backend	BDD, API métier (transactions), service d'auth, script de vérification de transaction, tests, doc d'API
Gabriel	Dev front	Front de paiement et dashboard de supervision (état des services, événements, alertes, avancement de la reprise), préparation du Clip Chrono
François	IA RAG + Red Team	Assistant RAG (Ollama + LangChain, indexation des procédures et de l'archi), scripts d'attaque sur le PC Attaquant, scénarios de chaos de jeudi

Gouvernance (pas de spécialiste, donc partagée) :

- Antoine : EBIOS RM + PCA/PRA, avec les infos techniques de Hugo.
- Gabriel : NIS2/DORA + pitch et `Pres.pptx`.
- François : assemblage du Dossier PDF, avec la contribution de chacun bien visible.
- Kevin : relecture et cohérence avec le démonstrateur.

Plan par jour

Le fil rouge de bout en bout doit tourner dès mardi, même moche, pour éviter une intégration tardive.

Lundi 19 octobre : architecture et EBIOS

- Valider l'architecture : services, flux, réseaux.
- Choisir la stack (suggestion : Python/FastAPI partout pour aller vite, ce qui simplifie ML et remédiation).
- Choisir l'attaque principale (voir la section suivante).
- Ébaucher la matrice EBIOS avec tout le monde, pour la validation des coachs.
- Créer le dépôt Git, fixer les conventions et vérifier le hotspot Windows 192.168.137.x.
- Vérifier la RAM du PC Hôte : sous 32 Go, prévoir un 3e PC pour Ollama.

Mardi : socle et premiers composants

- Socle déployable en une commande (Compose + Ansible).
- Microservices fonctionnels.
- Observabilité qui remonte métriques et logs.
- Premiers scripts d'attaque et premier jeu de données ML.
- EBIOS et NIS2/DORA rédigés.

Mercredi : intégration

- Tout brancher : WAF, logs, SIEM, ML, remédiation, dashboard.
- Premiers tests de restauration.
- RAG opérationnel.

Jeudi : recette et rendus

- Chaos engineering et au moins 5 passages de démo sans intervention.
- Tournage du clip en une prise.
- Dossier, Pres et zip (vérifier l'absence de secrets).
- Gel du code à 17h, remise avant 23h59.

Vendredi : jury

On ne code plus, on présente.

Suggestion de scénario d'attaque

Un flood HTTP contrôlé est facile à rendre reproductible et se prête bien à une réponse automatique.

1. Flood HTTP contrôlé depuis le PC Attaquant, qui sature l'API.
2. Les métriques et logs montrent un pic anormal.
3. Le ML détecte l'anomalie, la remédiation bloque l'IP (WAF ou pare-feu) et relance ou remet à l'échelle l'API.
4. Un script rejoue une transaction et valide le retour à la normale.

En bonus : une injection SQL bloquée par le WAF et tracée dans le SIEM. Il faudra alors expliquer que bloquer n'est pas corriger une compromission.

Risques à anticiper

- **RAM du PC Hôte** : Docker + Loki + Grafana + Ollama sur 16 Go, c'est très juste.
- **Intégration tardive** : tenir le fil rouge de bout en bout dès mardi, même moche.
- **Démo non reproductible** : à tester en boucle, puisque c'est le 0/5 qui menace.
- **Gouvernance bâclée** : elle doit citer notre architecture et nos essais réels.